

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Технологии разработки программного обеспечения
ОТЧЁТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ
РАБОТЫ №2

«Разработка клиент-серверного приложения, вычисляющего длину
минимального пути в неориентированном графе»

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Силиненко А. В.

«_____» _____ 2025 г.

Санкт-Петербург, 2025

Содержание

Введение	4
1 Спецификация требований	5
1.1 Общее описание задачи	5
1.2 Описание интерфейса	5
1.2.1 Входные данные	5
1.2.2 Выходные данные	6
1.2.3 Хранимые данные	6
1.3 Функциональные требования	7
1.3.1 Ввод данных	7
1.3.2 Проверка корректности данных	7
1.3.3 Содержательная часть	7
1.3.4 Вывод результатов	8
1.3.5 Завершение работы программы	8
1.4 Требования к тестированию	8
1.4.1 Тестирование по протоколу TCP	8
1.4.2 Тестирование по протоколу UDP	8
1.4.3 Ввод с клавиатуры	8
1.4.4 Ввод из файла	9
1.4.5 Обработка граничных значений	9
1.4.6 Обработка значений из середины области	9
2 Проектирование приложения	10
2.1 Перечень модулей	10
2.1.1 Модуль клиентской части	10
2.1.2 Модуль серверной части	10
2.2 Алгоритмы работы клиентской и серверной частей приложения при функционировании по протоколу UDP	10
2.3 Форматы структур данных, передаваемых между клиентской и серверной частями приложения	10
3 Разработка приложения	12
3.1 Используемая среда разработки	12
3.2 Используемый компилятор	12
3.3 Основные технологии, используемые в приложении	12
3.3.1 Механизм межпроцессного взаимодействия	12
3.3.2 Механизм мультимплексирования ввода/вывода	13
3.4 Пример запуска и работы программы	13
3.4.1 Запуск сервера	13
3.4.2 Запуск клиента	13
3.4.3 Результат	14

4	Тестирование	15
	Заключение	16
	Список литературы	17
A	Исходный код tcpclient.cpp	18
B	Исходный код tcpserver.cpp	29
C	Исходный код тестирующего приложения	38
D	Протоколы тестирования	47

Введение

Клиент-серверные приложения представляют собой архитектурную модель взаимодействия программного обеспечения, при которой одна часть системы (сервер) предоставляет ресурсы или услуги, а другая (клиент) запрашивает и использует их. Такой подход широко применяется в сетевых технологиях, веб-разработке, базах данных и распределённых системах. Взаимодействие между клиентом и сервером осуществляется посредством сетевых протоколов, наиболее распространёнными из которых являются TCP и UDP.

Цель работы: разработать клиент-серверное приложение, вычисляющее длину минимального пути в неориентированном графе.

Задачи работы:

- получение знаний об этапах разработки программного обеспечения;
- получение базовых знаний по разработке приложений в ОС Linux;
- получение знаний и практических навыков реализации межпроцессных взаимодействий с помощью механизма сокетов;
- получение знаний по работе с графами;
- разработка клиент-серверного приложения.

1 Спецификация требований

1.1 Общее описание задачи

Назначение: клиент-серверное приложение, вычисляющее длину минимального пути в неориентированном графе.

Язык реализации: C++

Архитектура: x64

Операционная система: Linux Ubuntu 24.04.3 LTS

1.2 Описание интерфейса

1.2.1 Входные данные

Данные, вводимые пользователем через клиентскую часть

1. Матрица размера $N \times M$, где целое число $6 \leq N \leq 40$ – количество вершин графа, целое число $M \geq 6$ – количество рёбер графа.
2. Целое число от 1 до N – индекс начальной вершины пути;
3. Целое число от 1 до N – индекс конечной вершины пути.

Входные данные клиента

1. Тип подключения TCP|UDP;
2. IP-адрес;
3. Порт серверной части
4. Ответ от сервера – строка, полученная через сокет;
5. Данные, вводимые пользователем с клавиатуры;
6. Данные, загруженные из файла.

Входные данные сервера

1. Тип подключения TCP|UDP;
2. Структура данных, которая содержит описание графа, отправленная клиентом через сокет.

1.2.2 Выходные данные

Выходные данные клиента

1. Структура данных, описывающих граф

Выходные данные сервера

1. Целое неотрицательное число – суммарный вес кратчайшего пути и список вершин минимального пути;
2. Сообщение об отсутствии пути между вершинами.

Информационные сообщения

1. Приглашение «Введите количество вершин в графе.»;
2. Приглашение «Введите матрицу инцидентности графа.»;
3. Приглашение «Введите номер начальной вершины.»;
4. Приглашение «Введите номер конечной вершины.»;
5. Ошибки:
 - «Использование: ./tcpclient <ip> <port> <tcp|udp> [file]»;
 - «Ошибка: не удалось открыть файл '<имя_файла>'»;
 - «Некорректный граф в файле»;
 - «Ошибка ввода. Повторите.»;
 - «Не удалось подключиться к серверу»;
 - «Ошибка отправки по ТСР»;
 - «Нет ответа от сервера»;
 - «Потеряна связь с сервером»;
 - «Сервер: ошибка обработки графа»;

1.2.3 Хранимые данные

На стороне сервера граф хранится в виде списков смежности, на стороне клиента граф хранится в виде матрицы инцидентности. Данные хранятся только на время обработки запроса и не сохраняются между сессиями.

1.3 Функциональные требования

1.3.1 Ввод данных

Клиентская часть поддерживает ввод данных (размерность, структуру, индексы вершин) с клавиатуры в интерактивном режиме и из текстового файла.

Сервер должен корректно принимать и интерпретировать передаваемые клиентом данные.

1.3.2 Проверка корректности данных

Клиент должен проверять корректность данных согласно требованиям:

- $6 \leq N \leq 40$, где N – количество вершин
- $M \geq 6$, где M – количество рёбер
- $1 \leq V_1 \leq N$, $1 \leq V_2 \leq N$, где V_1, V_2 – индексы стартовой и конечной вершины

При нарушении хотя бы 1 условия данные считаются некорректными.

1.3.3 Содержательная часть

Серверная часть

- поддерживает одновременную работу с не менее чем тремя клиентами;
- применяет алгоритм Дейкстры для поиска кратчайшего пути;
- возвращает только длину пути, перечень вершин кратчайшего пути или ошибку;

Клиентская часть

- поддерживает интерактивный ввод графа с клавиатуры и чтение из файла;
- отправляет запрос серверу по выбранному протоколу (TCP или UDP с подтверждением);
- получает от сервера строку с длиной пути и списком вершин кратчайшего маршрута;
- корректно обрабатывает ответ (включая случай отсутствия пути) и выводит его пользователю.

1.3.4 Вывод результатов

Результат работы – целое число, длина найденного кратчайшего пути и перечень вершин кратчайшего пути, или сообщение об отсутствии пути между вершинами.

1.3.5 Завершение работы программы

Клиентская часть завершает работу по команде `exit` (в интерактивном режиме). Сервер продолжает работу до получения сигнала завершения `kill`.

1.4 Требования к тестированию

Требуется разработать автоматические тесты, обеспечивающие тестирование разработанного клиент-серверного приложения. Тестирование должно выполняться автоматически без участия пользователя.

1.4.1 Тестирование по протоколу TCP

- Клиент должен успешно устанавливать соединение с сервером;
- Сервер должен успешно запускаться и принимать входящие подключения;
- Сервер должен выполнять вычисление длины минимального пути;
- Клиент корректно получает и отображает результат вычисления.

1.4.2 Тестирование по протоколу UDP

- Присутствует корректная передача данных без установления постоянного соединения;
- Повторная отправка данных клиентом при отсутствии подтверждения;
- Корректное завершение работы клиента при достижении максимального числа попыток передачи.

1.4.3 Ввод с клавиатуры

- Приложение должно корректно принимать данные о графе;
- Корректно введенные данные должны передаваться и обрабатываться на сервере.

1.4.4 Ввод из файла

- Приложение должно корректно принимать данные о графе из файла;
- Корректно считанные данные должны передаваться и обрабатываться на сервере.

1.4.5 Обработка граничных значений

- Приложение должно корректно работать при минимальном допустимом количестве вершин графа $N = 6$;
- Приложение должно корректно работать при максимальном допустимом количестве вершин графа $N = 40$;
- Клиент должен отклонять запрос когда количество вершин не входит в диапазон $6 \leq N \leq 40$ ($N = 41$ и $N = 5$).

1.4.6 Обработка значений из середины области

- Приложение должно корректно работать при количестве вершин графа $N = 20$;

2 Проектирование приложения

2.1 Перечень модулей

Приложение состоит из двух независимых исполняемых модулей: сервер и клиент.

2.1.1 Модуль клиентской части

Клиентская часть отвечает за приём входных данных, валидацию входных данных, передачу запроса на сервер, обработку ответа, обеспечение отказоустойчивости по протоколу UDP.

2.1.2 Модуль серверной части

Серверная часть отвечает за получение данных от клиента, валидацию входных данных, преобразование графа в списки смежности, вычисление кратчайшего пути, формирование ответа, обеспечение параллельной обработки клиентов.

2.2 Алгоритмы работы клиентской и серверной частей приложения при функционировании по протоколу UDP

Блок-схема работы алгоритмов приведена на рисунке 1.

2.3 Форматы структур данных, передаваемых между клиентской и серверной частями приложения

Передача осуществляется в текстовом виде:

От клиента к серверу:

```
V E
inc[0][0] inc[0][1] ... inc[0][E-1]
...
inc[V-1][0] ... inc[V-1][E-1]
from to
```

От сервера клиенту:

В случае ошибки:

Ответ:

0

В случае отсутствия ошибки:

1 <dist> <path_len> <v0> <v1> ... <vk>

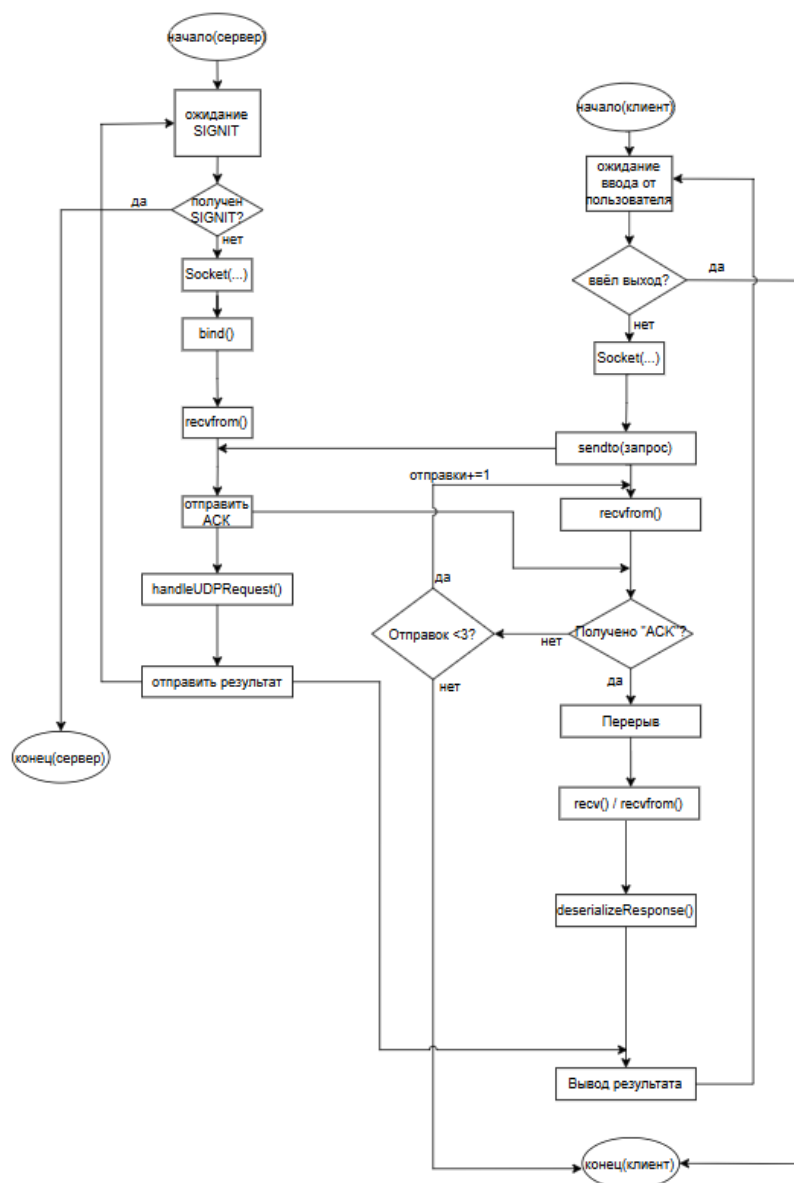


Рис. 1: Блок-схема работы алгоритмов

3 Разработка приложения

3.1 Используемая среда разработки

В качестве среды разработки использовалось приложение Code::Blocks на виртуальной машине Ubuntu 24.04.3 LTS, запущенной в VirtualBox. Запуск и отладка выполнялись из стандартного терминала Ubuntu с использованием средств командной строки.

3.2 Используемый компилятор

Сборка и компиляция проекта выполнялись вручную через командную строку. Использован компилятор GCC(GNU Compiler Collection):

```
g++ tcpserver.cpp -o tcpserver
g++ tcpclient.cpp -o tcpclient
```

3.3 Основные технологии, используемые в приложении

3.3.1 Механизм межпроцессного взаимодействия

Взаимодействие между клиентом и сервером реализовано с использованием сетевых сокетов TCP/IP на основе протоколов TCP и UDP. Обмен данными осуществляется по протоколу IPv4. Для установления соединения и передачи данных используются следующие основные системные вызовы:

- `socket()` — создаёт сокет;
- `bind()` — привязывает сокет сервера к локальному IP-адресу и порту;
- `listen()` — переводит TCP-сокет сервера в режим ожидания;
- `accept()` — принимает входящее TCP-соединение от клиента и создаёт новый сокет для обмена данными с этим клиентом;
- `connect()` — инициирует соединение с сервером на стороне TCP-клиента;
- `send()`, `recv()` — используются для передачи и приёма данных по TCP-соединению;
- `sendto()`, `recvfrom()` — применяются для обмена дейтаграммами по UDP; позволяют указывать адрес получателя/отправителя;
- `close()` — закрывает сокет.

При работе по протоколу UDP клиент реализует механизм подтверждения доставки: после отправки данных он ожидает получения служебного сообщения «АСК» от сервера, а затем — самого результата. При отсутствии подтверждения в течение заданного времени (3 секунды) выполняется повторная отправка (до 3 попыток).

3.3.2 Механизм мультиплексирования ввода/вывода

Чтобы сервер мог одновременно обслуживать несколько клиентов, используется механизм мультиплексирования ввода/вывода.

Этот механизм реализуется с помощью `select()`, который позволяет серверу прослушивать сразу несколько сокетов: как входящие подключения, так и данные от уже подключённых клиентов.

3.4 Пример запуска и работы программы

3.4.1 Запуск сервера

TCP:

```
./tcpserver 54200 tcp
TCP-сервер запущен на порту 54200
```

UDP:

```
./tcpserver 54200 udp
UDP-сервер запущен на порту 54200
```

3.4.2 Запуск клиента

Ручной ввод:

```
./tcpclient 127.0.0.1 54200 tcp
Клиент запущен. Введите 'exit' в любой момент для выхода.
```

--- Новый запрос ---

Введите количество вершин (V) или 'exit': 6

Введите количество рёбер (E): 6

Введите матрицу инцидентности (6 строк, 6 столбцов):

```
6 1 0 0 0 0
0 1 2 0 0 0
0 0 2 3 0 0
0 0 0 3 4 0
0 0 0 0 4 5
```

6 0 0 0 0 5

Введите начальную вершину (0-based): 0

Введите конечную вершину (0-based): 3

Ввод из файла:

```
./tcpclient 127.0.0.1 54200 tcp graph.txt
```

3.4.3 Результат

Кратчайший путь: 0 -> 1 -> 2 -> 3

Длина: 6

Исходный код программы приведён в приложениях А, В.

4 Тестирование

Тестирующее приложение разработано с использованием shell-скриптов и языка сценариев Exрест, предназначенного для автоматизации интерактивных консольных программ.

Exрест является расширением языка Tcl и широко применяется в операционных системах семейства Linux для автоматического ввода данных в консольные приложения, анализа вывода программ и тестирования клиент-серверных приложений без участия пользователя.

Разработаны тесты в следующем объёме:

- Работа приложения по протоколу TCP;
- Работа приложения по протоколу UDP, включая работу клиента при недоступности сервера;
- Ввод исходных данных с клавиатуры;
- Ввод исходных данных из файла;
- Работа приложения при задании графа, количество вершин которого близко к граничным значениям области допустимых значений (ОДЗ):
 - граф с количеством вершин, на единицу меньше нижней границы ОДЗ;
 - граф с количеством вершин, равным нижней границе ОДЗ;
 - граф с количеством вершин, равным верхней границе ОДЗ;
 - граф с количеством вершин, на единицу больше верхней границы ОДЗ;
- Работа приложения при задании значений количества вершин из середины ОДЗ.

Исходный код тестирующего приложения приведён в приложении С.

Проведено тестирование приложения с использованием разработанных тестов. Все тесты успешно выполнены. Протоколы тестирования приведены в приложении D.

Заключение

В результате выполнения лабораторной работы достигнута цель - разработано клиент-серверное приложение, вычисляющее длину минимального пути в неориентированном графе.

Выполнены поставленные задачи. Получены базовые знания по разработке приложений в ОС Linux. Получены знания и практические навыки реализации межпроцессных взаимодействий с помощью механизма сокетов, включая работу с TCP и UDP. Получены знания по работе с графами, реализован алгоритм Дейкстры для поиска кратчайшего пути. Разработаны автоматические тесты, тестирующие клиент-серверное приложение. Они обеспечивают проверку работы приложения по протоколам TCP, UDP, ввода графа с клавиатуры и из файла и обработку граничных значений.

Разработанное приложение корректно обрабатывает входные данные, выполняет вычисления на серверной стороне и передаёт результаты клиенту по сети.

Список литературы

- [1] Силиненко А. В. Лабораторная работа №2. Разработка клиент-серверного приложения. СПбПУ, 2025. URL: <https://dl.spbstu.ru/course/view.php?id=7420>
- [2] Сообщество Ubuntu. – Текст: электронный // help.ubuntu.ru: [сайт]. – URL: <https://help.ubuntu.ru/wiki/главная> (дата обращения: 20.09.2025)
- [3] Итмо. – Текст: электронный // <https://neerc.ifmo.ru>: [сайт]. – URL: <https://neerc.ifmo.ru/wiki/index.php?title=АлгоритмДейкстры> (дата обращения: 15.11.2025)

А Исходный код tcpclient.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <sstream>
5  #include <fstream>
6  #include <cstring>
7  #include <cstdlib>
8  #include <unistd.h>
9  #include <sys/socket.h>
10 #include <netinet/in.h>
11 #include <arpa/inet.h>
12 #include <chrono>
13 #include <thread>
14 #include <csignal>
15 #include <limits>
16 #include <fcntl.h>
17 #include <sys/select.h>
18 using namespace std;
19
20 const int MAX_RETRIES = 3;           // Максимум попыток
    отправки по UDP
21 const int TIMEOUT_SEC = 3;           // Таймаут ожидания ACK
22 const int MAX_VERTICES = 40;
23 const int MIN_VERTICES = 6;
24
25 enum Protocol { TCP, UDP };
26
27 // Проверка корректности графа аналогично( серверу)
28 bool validateGraph(int V, int E, const vector<vector<
    int>>& inc) {
29     if (V < MIN_VERTICES || V > MAX_VERTICES)
30         return false;
31     if (E < MIN_VERTICES) return false;
32     if ((int)inc.size() != V) return false;
33     for (const auto& row : inc)
34         if ((int)row.size() != E) return false;
35     for (int j = 0; j < E; ++j) {
36         int cnt = 0;
37         int weight = -1;
38         for (int i = 0; i < V; ++i) {
```

```

38         if (inc[i][j] < 0) return false
;
39         if (inc[i][j] > 0) {
40             if (weight == -1)
weight = inc[i][j];
41             else if (inc[i][j] !=
weight) return false;
42             cnt++;
43         }
44     }
45     if (cnt != 2) return false;
46 }
47 return true;
48 }
49
50 // Преобразует входные данные в строку для отправки на сервер
51 string serializeRequest(int V, int E, const vector<
vector<int>>& inc, int from, int to) {
52     ostringstream oss;
53     oss << V << "□" << E << "\n";
54     for (int i = 0; i < V; ++i) {
55         for (int j = 0; j < E; ++j) {
56             oss << inc[i][j] << "□";
57         }
58         oss << "\n";
59     }
60     oss << from << "□" << to;
61     return oss.str();
62 }
63
64 // Парсит ответ сервера: извлекает расстояние и путь
65 bool deserializeResponse(const string& data, int&
distance, vector<int>& path) {
66     istringstream iss(data);
67     bool success;
68     if (!(iss >> success)) return false;
69     if (!success) return false;
70     int len;
71     if (!(iss >> distance >> len)) return false;
72     path.resize(len);
73     for (int i = 0; i < len; ++i)
74         if (!(iss >> path[i])) return false;
75     return true;

```

```

76 }
77
78 // Надёжная отправка по UDP с подтверждением (ACK)
79 // Повторяет отправку до MAX_RETRIES раз при отсутствии ACK
80 bool sendUDPWithAck(int sock, const sockaddr_in&
    serverAddr, const string& msg) {
81     for (int attempt = 0; attempt < MAX_RETRIES; ++
        attempt) {
82         if (sendto(sock, msg.c_str(), msg.size
            (()), 0, (const sockaddr*)&serverAddr, sizeof(
                serverAddr)) < 0) {
83             continue;
84         }
85         fd_set readfds;
86         FD_ZERO(&readfds);
87         FD_SET(sock, &readfds);
88         struct timeval tv = {TIMEOUT_SEC, 0};
89         int activity = select(sock + 1, &
            readfds, nullptr, nullptr, &tv);
90         if (activity > 0 && FD_ISSET(sock, &
            readfds)) {
91             char ackBuf[4] = {0};
92             recvfrom(sock, ackBuf, sizeof(
                ackBuf) - 1, 0, nullptr, nullptr);
93             if (string(ackBuf) == "ACK") {
94                 return true;
95             }
96         }
97     }
98     return false;
99 }
100
101 // Простая отправка по TCP проверка(полноты отправки)
102 bool sendTCP(int sock, const string& msg) {
103     return send(sock, msg.c_str(), msg.size(), 0)
        == (ssize_t)msg.size();
104 }
105
106 // Приём данных по TCP
107 string recvTCP(int sock) {
108     char buffer[4096] = {0};
109     ssize_t n = recv(sock, buffer, sizeof(buffer) -
        1, 0);

```

```

110         if (n <= 0) return "";
111         return string(buffer, n);
112     }
113
114     // Главная функция клиента:
115     // - принимает IP, порт, протокол и опционально() имя файла,
116     // - в режиме файла читает граф из файла и отправляет запрос,
117     // - в интерактивном режиме запрашивает данные у пользователя.
118     int main(int argc, char* argv[]) {
119         if (argc < 4) {
120             cerr << "Использование: ./tcpclient <ip> <port> <tcp|udp> [file]\n";
121             return 1;
122         }
123         string ip = argv[1];
124         int port = stoi(argv[2]);
125         string protoStr = argv[3];
126         Protocol proto = (protoStr == "tcp") ? TCP :
UDP;
127         bool useFile = (argc > 4);
128         if (useFile) {
129             // ===== РЕЖИМ ФАЙЛА =====
130             string filename = argv[4];
131             ifstream file(filename);
132             if (!file.is_open()) {
133                 cerr << "Ошибка: не удалось
открыть файл'" << filename << "'\n";
134                 return 1;
135             }
136             int V, E, from, to;
137             file >> V >> E;
138             vector<vector<int>> inc(V, vector<int>(
E));
139             for (int i = 0; i < V; ++i)
140                 for (int j = 0; j < E; ++j)
141                     file >> inc[i][j];
142             file >> from >> to;
143             file.close();
144             if (!validateGraph(V, E, inc)) {
145                 cerr << "Некорректный граф в
файле\n";
146                 return 1;
147             }

```

```

148         string msg = serializeRequest(V, E, inc
, from, to);
149         if (proto == TCP) {
150             int sock = socket(AF_INET,
SOCK_STREAM, 0);
151             if (sock < 0) { perror("socket"
); return 1; }
152             sockaddr_in serverAddr{};
153             serverAddr.sin_family = AF_INET
;
154             serverAddr.sin_port = htons(
port);
155             inet_pton(AF_INET, ip.c_str(),
&serverAddr.sin_addr);
156             if (connect(sock, (sockaddr*)&
serverAddr, sizeof(serverAddr)) < 0) {
157                 cerr << "Не удалось
подключиться к серверу\n";
158                 close(sock);
159                 return 1;
160             }
161             if (!sendTCP(sock, msg)) {
162                 cerr << "Ошибка отправки\
n";
163                 close(sock);
164                 return 1;
165             }
166             string response = recvTCP(sock)
;
167             close(sock);
168             if (response.empty()) {
169                 cerr << "Нет ответа от
сервера\n";
170                 return 1;
171             }
172             int distance;
173             vector<int> path;
174             if (!deserializeResponse(
response, distance, path)) {
175                 cout << "Сервер: ошибка
обработки графа\n";
176             } else {

```

```

177         cout << "Кратчайший_путь:"
178         _";
179         for (size_t i = 0; i <
180             path.size(); ++i) {
181             if (i > 0) cout
182                 << "_->_";
183             cout << path[i]
184             ];
185             }
186             cout << "\Длина:_" <<
187             distance << endl;
188         }
189         } else { // UDP
190             int sock = socket(AF_INET,
191                 SOCK_DGRAM, 0);
192             if (sock < 0) { perror("socket"
193 ); return 1; }
194             sockaddr_in serverAddr{};
195             serverAddr.sin_family = AF_INET
196             ;
197             serverAddr.sin_port = htons(
198             port);
199             inet_pton(AF_INET, ip.c_str(),
200             &serverAddr.sin_addr);
201             if (!sendUDPWithAck(sock,
202             serverAddr, msg)) {
203                 cerr << "Потеряна_связь_
204                 с_сервером\n";
205                 close(sock);
206                 return 1;
207             }
208             char respBuf[4096] = {0};
209             if (recv(sock, respBuf, sizeof(
210             respBuf) - 1, 0) <= 0) {
211                 cerr << "Нет_ответа_от_
212                 сервера\n";
213                 close(sock);
214                 return 1;
215             }
216             int distance;
217             vector<int> path;
218             if (!deserializeResponse(string
219             (respBuf), distance, path)) {

```

```

205         cout << "Сервер: _ошибка_
обработки_графа\n";
206     } else {
207         cout << "Кратчайший_путь:
_";
208         for (size_t i = 0; i <
path.size(); ++i) {
209             if (i > 0) cout
<< "_->_";
210             cout << path[i
];
211         }
212         cout << "\Длина:_ " <<
distance << endl;
213     }
214     close(sock);
215 }
216 return 0;
217 } else {
218     // ===== ИНТЕРАКТИВНЫЙ РЕЖИМ
=====
219     cout << "Клиент_запущен._Введите_'exit'_в_
любой_момент_для_выхода.\n";
220     while (true) {
221         cout << "\n---_Новый_запрос_---\n
n";
222         cout << "Введите_количество_
вершин_(V)_или_'exit':_";
223         string input;
224         cin >> input;
225         if (input == "exit") {
226             cout << "Завершение_
работы_клиента.\n";
227             break;
228         }
229         try {
230             int V = stoi(input);
231             int E;
232             cout << "Введите_
количество_рёбер_(E):_";
233             cin >> E;
234             vector<vector<int>> inc
(V, vector<int>(E));

```



```

235                                     cout << "Введите_матрицу_
инцидентности_(" << V << "_строк,_" << E << "_столбцов):\n";
236                                     for (int i = 0; i < V;
++i)
237                                     for (int j = 0; j < E;
++j)
238                                     cin >> inc[i][j];
239                                     int from, to;
240                                     cout << "Введите_
начальную_вершину_(0-based):_"; cin >> from;
241                                     cout << "Введите_
конечную_вершину_(0-based):_"; cin >> to;
242                                     if (!validateGraph(V, E
, inc)) {
243                                     cerr << "
Некорректный_граф!_(V>=6, _E>=6, _матрица_инцидентности)\n";
244                                     continue;
245                                     }
246                                     string msg =
serializeRequest(V, E, inc, from, to);
247                                     if (proto == TCP) {
248                                     int sock =
socket(AF_INET, SOCK_STREAM, 0);
249                                     if (sock < 0) {
perror("socket"); continue; }
250                                     sockaddr_in
serverAddr{};
251                                     serverAddr.
sin_family = AF_INET;
252                                     serverAddr.
sin_port = htons(port);
253                                     inet_pton(
AF_INET, ip.c_str(), &serverAddr.sin_addr);
254                                     if (connect(
sock, (sockaddr*)&serverAddr, sizeof(serverAddr)) <
0) {
255                                     cerr <<
"Не_удалось_подключиться_к_серверу\n";
256                                     close(
sock);
257                                     continue;

```

```

258                                     }
259                                     if (!sendTCP(
sock, msg)) {
260                                     cerr <<
261                                     "Ошибка_отправки_по_TCP\n";
262                                     close(
sock);
263                                     continue;
264                                     }
265                                     string response
= recvTCP(sock);
266                                     close(sock);
267                                     if (response.
empty()) {
268                                     cerr <<
269                                     "Нет_ответа_от_сервера\n";
270                                     continue;
271                                     }
272                                     int distance;
273                                     vector<int>
path;
274                                     if (!
deserializeResponse(response, distance, path)) {
275                                     cout <<
276                                     "Сервер:_ошибка_обработки_графа\n";
277                                     } else {
278                                     cout <<
279                                     "Кратчайший_путь:_";
280                                     for (
size_t i = 0; i < path.size(); ++i) {
281                                     if (i > 0) cout << "_->_";
282                                     cout << path[i];
283                                     }
284                                     cout <<
285                                     "\Длина\n:_" << distance << endl;
286                                     }
287                                     } else { // UDP
288                                     int sock =
socket(AF_INET, SOCK_DGRAM, 0);

```

```

284         perror("socket"); continue; }
285     sockaddr_in
serverAddr{};
286     serverAddr.
sin_family = AF_INET;
287     serverAddr.
sin_port = htons(port);
288     inet_pton(
AF_INET, ip.c_str(), &serverAddr.sin_addr);
289     if (!
sendUDPWithAck(sock, serverAddr, msg)) {
290         cerr <<
"Потеряна связь с сервером\n";
291         close(
sock);
292     continue;
293     }
294     char respBuf
[4096] = {0};
295     if (recv(sock,
respBuf, sizeof(respBuf) - 1, 0) <= 0) {
296         cerr <<
"Нет ответа от сервера\n";
297         close(
sock);
298     continue;
299     }
300     int distance;
301     vector<int>
path;
302     if (!
deserializeResponse(string(respBuf), distance, path)
) {
303         cout <<
"Сервер: ошибка обработки графа\n";
304     } else {
305         cout <<
"Кратчайший путь: ";
306         for (
size_t i = 0; i < path.size(); ++i) {

```

```

307         if (i > 0) cout << "□->□";
308
309         cout << path[i];
310
311     }
312     cout <<
313     "\Длинаn:□" << distance << endl;
314
315     }
316     close(sock);
317
318     } catch (...) {
319         cerr << "Ошибка□ввода.□
320 Повторите.\n";
321         cin.clear();
322         cin.ignore(
323         numeric_limits<streamsize>::max(), '\n');
324         continue;
325     }
326 }
327
328 }
329
330 return 0;
331
332 }

```

В Исходный код tcpserver.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <algorithm>
5  #include <cstring>
6  #include <unistd.h>
7  #include <sys/socket.h>
8  #include <netinet/in.h>
9  #include <arpa/inet.h>
10 #include <thread>
11 #include <csignal>
12 #include <sstream>
13 #include <atomic>
14 #include <sys/select.h>
15 #include <climits>
16 using namespace std;
17
18 const int MAX_VERTICES = 40;
19 const int MIN_VERTICES = 6;
20
21 // Глобальный флаг завершения работы сервера по сигналу
22 std::atomic<bool> serverRunning{true};
23
24 // Структура для представления графа как списка смежности
25 struct Graph {
26     int V;
27     vector<vector<pair<int, int>>> adj;
28 };
29
30 // Проверяет корректность матрицы инцидентности:
31 // - размеры соответствуют V и E,
32 // - каждое ребро соединяет ровно две вершины,
33 // - веса положительны и одинаковы для обеих инцидентных
34 //   вершин.
35 bool validateGraph(int V, int E, const vector<vector<
36     int>>& inc) {
37     if (V < MIN_VERTICES || V > MAX_VERTICES) return false;
38     if (E < MIN_VERTICES) return false;
39     if ((int)inc.size() != V) return false;
40     for (const auto& row : inc)
41         if ((int)row.size() != E) return false;
```

```

40 for (int j = 0; j < E; ++j) {
41     int cnt = 0;
42     int weight = -1;
43     for (int i = 0; i < V; ++i) {
44         if (inc[i][j] < 0) return false;
45         if (inc[i][j] > 0) {
46             if (weight == -1) weight = inc[
i][j];
47             else if (inc[i][j] != weight)
return false;
48             cnt++;
49         }
50     }
51     if (cnt != 2) return false;
52 }
53 return true;
54 }
55
56 // Преобразует матрицу инцидентности в список смежности для
    неориентированного графа
57 Graph incidenceToAdjacency(int V, int E, const vector<
    vector<int>>& inc) {
58     Graph g;
59     g.V = V;
60     g.adj.assign(V, vector<pair<int,int>>());
61     for (int j = 0; j < E; ++j) {
62         int u = -1, v = -1, w = 0;
63         for (int i = 0; i < V; ++i) {
64             if (inc[i][j] > 0) {
65                 if (u == -1) {
66                     u = i;
67                     w = inc[i][j];
68                 } else {
69                     v = i;
70                 }
71             }
72         }
73         if (u != -1 && v != -1) {
74             g.adj[u].push_back({v, w});
75             g.adj[v].push_back({u, w});
76         }
77     }
78     return g;

```

```

79 }
80
81 // Реализация алгоритма Дейкстры:
82 // возвращает пару расстояние(, путь) от src до dst.
83 // Если путь не существует, возвращает (-1, пустой вектор).
84 pair<int, vector<int>> dijkstra(const Graph& g, int src
85     , int dst) {
86     vector<int> dist(g.V, INT_MAX);
87     vector<int> parent(g.V, -1);
88     priority_queue<pair<int, int>, vector<pair<int, int>>,
89         greater<pair<int, int>>> pq;
90     dist[src] = 0;
91     pq.push({0, src});
92     while (!pq.empty()) {
93         int u = pq.top().second;
94         pq.pop();
95         for (auto& edge : g.adj[u]) {
96             int v = edge.first;
97             int weight = edge.second;
98             if (dist[u] + weight < dist[v]) {
99                 dist[v] = dist[u] + weight;
100                 parent[v] = u;
101                 pq.push({dist[v], v});
102             }
103         }
104     }
105     if (dist[dst] == INT_MAX) {
106         return {-1, {}};
107     }
108     vector<int> path;
109     for (int v = dst; v != -1; v = parent[v])
110         path.push_back(v);
111     reverse(path.begin(), path.end());
112     return {dist[dst], path};
113 }
114
115 // Обрабатывает один TCPзапрос- от клиента:
116 // - читает данные,
117 // - парсит их,
118 // - проверяет корректность,
119 // - вычисляет кратчайший путь или возвращает ошибку.
120 void handleTCPClient(int clientSock) {
121     char buffer[4096] = {0};

```

```

120 ssize_t n = recv(clientSock, buffer, sizeof(buffer) -
    1, 0);
121 if (n <= 0) {
122     close(clientSock);
123     return;
124 }
125 string data(buffer, n);
126 int V, E, from, to;
127 vector<vector<int>> inc;
128 istream iss(data);
129 if (!(iss >> V >> E)) {
130     string resp = "0"; // success=false
131     send(clientSock, resp.c_str(), resp.size(), 0);
132     close(clientSock);
133     return;
134 }
135 inc.assign(V, vector<int>(E));
136 for (int i = 0; i < V; ++i)
137 for (int j = 0; j < E; ++j)
138 if (!(iss >> inc[i][j])) {
139     string resp = "0";
140     send(clientSock, resp.c_str(), resp.size(), 0);
141     close(clientSock);
142     return;
143 }
144 if (!(iss >> from >> to)) {
145     string resp = "0";
146     send(clientSock, resp.c_str(), resp.size(), 0);
147     close(clientSock);
148     return;
149 }
150 if (!validateGraph(V, E, inc) || from < 0 || from >= V
    || to < 0 || to >= V) {
151     string resp = "0";
152     send(clientSock, resp.c_str(), resp.size(), 0);
153     close(clientSock);
154     return;
155 }
156 Graph g = incidenceToAdjacency(V, E, inc);
157 auto [dist, path] = dijkstra(g, from, to);
158 ostream oss;
159 if (dist == -1) {
160     oss << "0";

```



```

161 } else {
162     oss << "1_" << dist << "_" << path.size();
163     for (int v : path) oss << "_" << v;
164 }
165 string resp = oss.str();
166 send(clientSock, resp.c_str(), resp.size(), 0);
167 close(clientSock);
168 }
169
170 // Обработчик сигналов завершения
171 void signalHandler(int sig) {
172     serverRunning = false;
173 }
174
175 // Обработывает один UDPзапрос-:
176 // - читает входные данные,
177 // - сразу отправляет ACK для надёжной передачи,
178 // - выполняет логику поиска пути и возвращает результат.
179 void handleUDPRequest(int sock, const sockaddr_in&
    clientAddr, socklen_t addrLen, const string& data) {
180     int V, E, from, to;
181     vector<vector<int>> inc;
182     istringstream iss(data);
183     // Постепенная валидация
184     if (!(iss >> V >> E)) {
185         string resp = "0";
186         sendto(sock, resp.c_str(), resp.size(), 0, (
            sockaddr*)&clientAddr, addrLen);
187         return;
188     }
189     inc.assign(V, vector<int>(E));
190     for (int i = 0; i < V; ++i) {
191         for (int j = 0; j < E; ++j) {
192             if (!(iss >> inc[i][j])) {
193                 string resp = "0";
194                 sendto(sock, resp.c_str(), resp
                    .size(), 0, (sockaddr*)&clientAddr, addrLen);
195                 return;
196             }
197         }
198     }
199     if (!(iss >> from >> to)) {
200         string resp = "0";

```

```

201         sendto(sock, resp.c_str(), resp.size(), 0, (
sockaddr*)&clientAddr, addrLen);
202         return;
203     }
204     if (!validateGraph(V, E, inc) || from < 0 || from >= V
|| to < 0 || to >= V) {
205         string resp = "0";
206         sendto(sock, resp.c_str(), resp.size(), 0, (
sockaddr*)&clientAddr, addrLen);
207         return;
208     }
209     // Если всё корректно - обрабатываем
Graph g = incidenceToAdjacency(V, E, inc);
210     auto [dist, path] = dijkstra(g, from, to);
211     ostringstream oss;
212     if (dist == -1) {
213         oss << "0";
214     } else {
215         oss << "1_" << dist << "_" << path.size();
216         for (int v : path) oss << "_" << v;
217     }
218     string resp = oss.str();
219     sendto(sock, resp.c_str(), resp.size(), 0, (sockaddr*)&
clientAddr, addrLen);
220 }
221
222 // Главная функция сервера:
223 // - принимает аргументы: порт и протокол (tcp/udp),
224 // - настраивает обработку сигналов,
225 // - запускает соответствующий серверный цикл.
226 int main(int argc, char* argv[]) {
227     if (argc < 3) {
228         cerr << "Использование: ./tcpserver_<port>_<tcp|
udp>\n";
229         return 1;
230     }
231     int port = stoi(argv[1]);
232     string proto = argv[2];
233     bool useTCP = (proto == "tcp");
234     bool useUDP = (proto == "udp");
235     if (!useTCP && !useUDP) {
236         cerr << "Поддерживаются только 'tcp' или 'udp'\n";
237         return 1;
238     }

```

```

239 }
240 struct sigaction sa;
241 sa.sa_handler = signalHandler;
242 sigemptyset(&sa.sa_mask);
243 sa.sa_flags = 0;
244 sigaction(SIGINT, &sa, nullptr);
245 sigaction(SIGTERM, &sa, nullptr);
246 if (useTCP) {
247     // === TCP сервер -: использует select() для
    прерываемого ожидания подключений ===
248     int serverSock = socket(AF_INET, SOCK_STREAM,
    0);
249     if (serverSock < 0) {
250         perror("socket");
251         return 1;
252     }
253     int opt = 1;
254     setsockopt(serverSock, SOL_SOCKET, SO_REUSEADDR
    , &opt, sizeof(opt));
255     sockaddr_in serverAddr{};
256     serverAddr.sin_family = AF_INET;
257     serverAddr.sin_addr.s_addr = INADDR_ANY;
258     serverAddr.sin_port = htons(port);
259     if (bind(serverSock, (sockaddr*)&serverAddr,
    sizeof(serverAddr)) < 0) {
260         perror("bind");
261         close(serverSock);
262         return 1;
263     }
264     if (listen(serverSock, 3) < 0) {
265         perror("listen");
266         close(serverSock);
267         return 1;
268     }
269     cout << "TCP сервер - запущен на порту " << port <<
    endl;
270     while (serverRunning) {
271         fd_set readfds;
272         FD_ZERO(&readfds);
273         FD_SET(serverSock, &readfds);
274         struct timeval timeout;
275         timeout.tv_sec = 1;
276         timeout.tv_usec = 0;

```

```

277         int activity = select(serverSock + 1, &
readfds, nullptr, nullptr, &timeout);
278         if (activity < 0) {
279             if (errno == EINTR) continue;
280             perror("select");
281             break;
282         }
283         if (activity == 0) {
284             continue;
285         }
286         sockaddr_in clientAddr;
287         socklen_t addrLen = sizeof(clientAddr);
288         int clientSock = accept(serverSock, (
sockaddr*)&clientAddr, &addrLen);
289         if (clientSock < 0) {
290             if (!serverRunning) break;
291             continue;
292         }
293         thread(handleTCPClient, clientSock).
detach();
294     }
295     close(serverSock);
296 } else if (useUDP) {
297     // == UDP сервер -: принимает пакеты, отправляет ACK,
обработывает запрос ==
298     int serverSock = socket(AF_INET, SOCK_DGRAM, 0)
;
299     if (serverSock < 0) {
300         perror("socket");
301         return 1;
302     }
303     sockaddr_in serverAddr{};
304     serverAddr.sin_family = AF_INET;
305     serverAddr.sin_addr.s_addr = INADDR_ANY;
306     serverAddr.sin_port = htons(port);
307     if (bind(serverSock, (sockaddr*)&serverAddr,
sizeof(serverAddr)) < 0) {
308         perror("bind");
309         close(serverSock);
310         return 1;
311     }
312     cout << "UDP сервер- запущен на порту" << port <<
endl;

```

```

313     while (serverRunning) {
314         char buffer[4096] = {0};
315         sockaddr_in clientAddr;
316         socklen_t addrLen = sizeof(clientAddr);
317         ssize_t n = recvfrom(serverSock, buffer
, sizeof(buffer) - 1, 0,
318         (sockaddr*)&clientAddr, &addrLen);
319         if (n <= 0) continue;
320         string data(buffer, n);
321         // Отправить ACK немедленно
322         const char* ack = "ACK";
323         sendto(serverSock, ack, 3, 0, (sockaddr
*)&clientAddr, addrLen);
324         // Обработать запрос и отправить ответ
325         handleUDPRequest(serverSock, clientAddr
, addrLen, data);
326     }
327     close(serverSock);
328 }
329 cout << "Сервер_завершил_работу.\n";
330 return 0;
331 }

```

С Исходный код тестирующего приложения

```
1 #!/bin/bash
2
3 set -e
4 echo "Запуск всех тестов..."
5 ./test_tcp_valid.sh
6 ./test_udp_valid.sh
7 ./test_udp_no_server.sh
8 ./test_tcp_interactive.exp
9 ./test_file_input.sh
10 ./test_boundary_v5.sh
11 ./test_boundary_V101.sh
12 ./test_V6.sh
13 ./test_boundary_V40.sh
14 ./test_V20.sh
15 echo "Все тесты пройдены!"
16
17 #!/bin/bash
18 cat > graph_V5.txt <<EOF
19 5 6
20 1 0 0 0 0 0
21 1 2 0 0 0 0
22 0 2 3 0 0 0
23 0 0 3 4 0 0
24 0 0 0 4 5 0
25 0 5
26 EOF
27
28 PROJ_DIR="$HOME/trpo/lab2"
29 PORT=8898
30 SERVER_BIN="$PROJ_DIR/tcpserver"
31 CLIENT_BIN="$PROJ_DIR/tcpclient"
32
33 g++ -o "$SERVER_BIN" "$PROJ_DIR/tcpserver.cpp" -
    lpthread -std=c++17
34 g++ -o "$CLIENT_BIN" "$PROJ_DIR/tcpclient.cpp" -
    lpthread -std=c++17
35
36 "$SERVER_BIN" "$PORT" tcp &
37 SERVER_PID=$!
38 sleep 1
39
```

```

40 OUTPUT=$(timeout 10 "$CLIENT_BIN" 127.0.0.1 "$PORT" tcp
    graph_V5.txt 2>&1)
41 EXIT_CODE=$?
42 kill "$SERVER_PID" 2>/dev/null
43 wait "$SERVER_PID" 2>/dev/null
44
45 if [ $EXIT_CODE -eq 1 ] && echo "$OUTPUT" | grep -q "
    Некорректный граф"; then
46 echo "тест: граф с количеством вершин, на единицу меньше 0ДЗ
    (5) - успешно"
47 exit 0
48 else
49 echo "V=5: должен быть отклонён"
50 echo "$OUTPUT"
51 exit 1
52 fi
53
54 #!/bin/bash
55
56 PROJ_DIR="$HOME/trpo/lab2"
57 PORT=8915
58 SERVER_BIN="$PROJ_DIR/tcpserver"
59 CLIENT_BIN="$PROJ_DIR/tcpclient"
60 TEST_FILE="graph_V40.txt"
61
62 cat > "$TEST_FILE" <<EOF
63 40 40
64 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0 2
65 1 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0
66 0 2 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0
67 0 0 3 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0
68 0 0 0 4 5 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0
69 0 0 0 0 5 6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0
70 0 0 0 0 0 6 7 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0
71 0 0 0 0 0 0 7 8 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 0 0 0

```

72	0	0	0	0	0	0	0	8	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
73	0	0	0	0	0	0	0	0	9	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
74	0	0	0	0	0	0	0	0	0	1	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
75	0	0	0	0	0	0	0	0	0	0	2	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
76	0	0	0	0	0	0	0	0	0	0	3	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
77	0	0	0	0	0	0	0	0	0	0	0	4	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
78	0	0	0	0	0	0	0	0	0	0	0	0	5	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
79	0	0	0	0	0	0	0	0	0	0	0	0	0	6	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
80	0	0	0	0	0	0	0	0	0	0	0	0	0	0	7	8	0	0	0	0	0	0	0	0	0	0	0	0	0	0
81	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8	9	0	0	0	0	0	0	0	0	0	0	0	0	0
82	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	1	0	0	0	0	0	0	0	0	0	0	0	0
83	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	2	0	0	0	0	0	0	0	0	0	0	0	0
84	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	3	0	0	0	0	0	0	0	0	0	0	0	0
85	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	4	0	0	0	0	0	0	0	0	0	0	0	0
86	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4	5	0	0	0	0	0	0	0	0	0	0	0	0
87	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	5	6	0	0	0	0	0	0	0	0	0	0	0	0
88	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	6	7	0	0	0	0	0	0	0	0	0	0	0	0
89	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	7	8	0	0	0	0	0	0	0	0	0	0	0	0
90	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8	9	0	0	0	0	0	0	0	0	0	0	0	0
91	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	1	0	0	0	0	0	0	0	0	0	0	0	0
92	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1


```

93 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    2 3 0 0 0 0 0 0 0 0 0 0 0 0
94 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 3 4 0 0 0 0 0 0 0 0 0 0 0
95 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 4 5 0 0 0 0 0 0 0 0 0 0
96 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 5 6 0 0 0 0 0 0 0 0 0
97 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 6 7 0 0 0 0 0 0 0 0
98 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 7 8 0 0 0 0 0 0 0
99 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 8 9 0 0 0 0 0 0
100 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 9 1 0 0 0 0 0
101 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 1 2 0 0 0 0
102 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 2 3 0 0 0
103 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    0 0 0 0 0 0 0 0 0 0 3 2 0 0
104 0 39
105 EOF
106
107 g++ -o "$SERVER_BIN" "$PROJ_DIR/tcpserver.cpp" -
    lpthread -std=c++17
108 g++ -o "$CLIENT_BIN" "$PROJ_DIR/tcpclient.cpp" -
    lpthread -std=c++17
109
110 "$SERVER_BIN" "$PORT" tcp &
111 SERVER_PID=$!
112 sleep 1
113
114 OUTPUT=$(timeout 10 "$CLIENT_BIN" 127.0.0.1 "$PORT" tcp
    "$TEST_FILE" 2>&1)
115 EXIT_CODE=$?
116
117 rm -f "$TEST_FILE"
118 kill "$SERVER_PID" 2>/dev/null
119 wait "$SERVER_PID" 2>/dev/null
120

```

```

121 if [ $EXIT_CODE -eq 0 ] && echo "$OUTPUT" | grep -q "
    Кратчайший_путь" && echo "$OUTPUT" | grep -q "Длина";
    then
122 echo "тест:_граф_с_количеством_вершин,_равным_верхней_границе_
    ОДЗ_(40)_-_успешно"
123 exit 0
124 else
125 echo "V=40:_ошибка"
126 echo "Вывод:_$OUTPUT"
127 exit 1
128 fi
129
130 #!/bin/bash
131
132 PORT=8903
133 PROJ_DIR="$HOME/trpo/lab2"
134 SERVER_BIN="$PROJ_DIR/tcpserver"
135 CLIENT_BIN="$PROJ_DIR/tcpclient"
136 TEST_FILE="graph_V101.txt"
137
138 cat > "$TEST_FILE" <<EOF
139 41 6
140 1 0 0 0 0 0
141 1 2 0 0 0 0
142 0 2 3 0 0 0
143 0 0 3 4 0 0
144 0 0 0 4 5 0
145 0 0 0 0 5 6
146 0 5
147 EOF
148
149 g++ -o "$SERVER_BIN" "$PROJ_DIR/tcpserver.cpp" -
    lpthread -std=c++17
150 g++ -o "$CLIENT_BIN" "$PROJ_DIR/tcpclient.cpp" -
    lpthread -std=c++17
151
152 "$SERVER_BIN" "$PORT" tcp &
153 SERVER_PID=$!
154 sleep 1
155
156 if ! kill -0 "$SERVER_PID" 2>/dev/null; then
157 echo "Сервер_не_запустился"
158 rm -f "$TEST_FILE"

```

```

159 exit 1
160 fi
161
162 OUTPUT=$(timeout 10 "$CLIENT_BIN" 127.0.0.1 "$PORT" tcp
    "$TEST_FILE" 2>&1)
163 EXIT_CODE=$?
164
165 rm -f "$TEST_FILE"
166 kill "$SERVER_PID" 2>/dev/null
167 wait "$SERVER_PID" 2>/dev/null
168
169 if [ $EXIT_CODE -eq 1 ] && echo "$OUTPUT" | grep -q "
    Некорректный_граф"; then
170 echo "тест: _граф_с_количеством_вершин_на_единицу_больше_ОДЗ_
    (41) _-_успешно"
171 exit 0
172 else
173 echo "V=41: _должен_быть_отклонён,_но_не_был"
174 echo "Код_выхода: _$EXIT_CODE"
175 echo "Вывод: _$OUTPUT"
176 exit 1
177 fi
178
179 #!/bin/bash
180
181 PROJ_DIR="$HOME/trpo/lab2"
182 PORT=8897
183 SERVER_BIN="$PROJ_DIR/tcpserver"
184 CLIENT_BIN="$PROJ_DIR/tcpclient"
185
186 g++ -o "$SERVER_BIN" "$PROJ_DIR/tcpserver.cpp" -
    lpthread -std=c++17
187 g++ -o "$CLIENT_BIN" "$PROJ_DIR/tcpclient.cpp" -
    lpthread -std=c++17
188
189 "$SERVER_BIN" "$PORT" tcp &
190 SERVER_PID=$!
191 sleep 1
192
193 cat > test_file_input.txt <<EOF
194 6 6
195 6 1 0 0 0 0
196 0 1 2 0 0 0

```

```

197 0 0 2 3 0 0
198 0 0 0 3 4 0
199 0 0 0 0 4 5
200 6 0 0 0 0 5
201 1 5
202 EOF
203
204 OUTPUT=$(timeout 10 "$CLIENT_BIN" 127.0.0.1 "$PORT" tcp
    test_file_input.txt 2>&1)
205 EXIT_CODE=$?
206 rm -f test_file_input.txt
207 kill "$SERVER_PID" 2>/dev/null
208 wait "$SERVER_PID" 2>/dev/null
209
210 if [ $EXIT_CODE -eq 0 ] && echo "$OUTPUT" | grep -q "
    Кратчайший_путь"; then
211 echo "тест:_ввод_исходных_данных_из_файла_-_успешно"
212 exit 0
213 else
214 echo "Файл:_не_работает"
215 echo "$OUTPUT"
216 exit 1
217 fi
218
219 #!/bin/bash
220
221 PORT=8888
222 PROJ_DIR="$HOME/trpo/lab2"
223 SERVER_BIN="$PROJ_DIR/tcpserver"
224 CLIENT_BIN="$PROJ_DIR/tcpclient"
225
226 g++ -o "$SERVER_BIN" "$PROJ_DIR/tcpserver.cpp" -
    lpthread -std=c++17
227 g++ -o "$CLIENT_BIN" "$PROJ_DIR/tcpclient.cpp" -
    lpthread -std=c++17
228
229 "$SERVER_BIN" "$PORT" tcp &
230 SERVER_PID=$!
231 sleep 1
232
233 cat > test_graph_valid.txt <<EOF
234 6 6
235 6 1 0 0 0 0

```

```

236 0 1 2 0 0 0
237 0 0 2 3 0 0
238 0 0 0 3 4 0
239 0 0 0 0 4 5
240 6 0 0 0 0 5
241 1 5
242 EOF
243
244 OUTPUT=$(timeout 10 "$CLIENT_BIN" 127.0.0.1 "$PORT" tcp
    test_graph_valid.txt 2>&1)
245 EXIT_CODE=$?
246
247 rm -f test_graph_valid.txt
248
249 if [ $EXIT_CODE -eq 124 ]; then
250 echo "TCP: клиент завис"
251 kill "$SERVER_PID" 2>/dev/null
252 exit 1
253 fi
254
255 if echo "$OUTPUT" | grep -q "Кратчайший путь" && echo "
    $OUTPUT" | grep -q "Длина"; then
256 echo "тест: работа приложения по протоколу TCP - успешно"
257 RESULT=0
258 else
259 echo "TCP: ожидаемый вывод не найден"
260 echo "Вывод: $OUTPUT"
261 RESULT=1
262 fi
263
264 kill "$SERVER_PID" 2>/dev/null
265 wait "$SERVER_PID" 2>/dev/null
266 exit $RESULT
267
268 #!/bin/bash
269
270 PORT=8889
271 PROJ_DIR="$HOME/trpo/lab2"
272 CLIENT_BIN="$PROJ_DIR/tcpclient"
273
274 g++ -o "$CLIENT_BIN" "$PROJ_DIR/tcpclient.cpp" -
    lpthread -std=c++17
275

```

```

276 cat > test_graph_small.txt <<EOF
277 6 6
278 6 1 0 0 0 0
279 0 1 2 0 0 0
280 0 0 2 3 0 0
281 0 0 0 3 4 0
282 0 0 0 0 4 5
283 6 0 0 0 0 5
284 1 5
285 EOF
286
287 OUTPUT=$(timeout 15 "$CLIENT_BIN" 127.0.0.1 "$PORT" udp
      test_graph_small.txt 2>&1)
288 EXIT_CODE=$?
289
290 rm -f test_graph_small.txt
291
292 if [ $EXIT_CODE -eq 124 ]; then
293 echo "UDP: клиент завис вместо таймаута"
294 exit 1
295 elif [ $EXIT_CODE -eq 1 ] && echo "$OUTPUT" | grep -q "
      Потеряна связь с сервером"; then
296 echo "тест: работа клиента при недоступности сервера -
      успешно"
297 exit 0
298 else
299 echo "UDP: сервер недоступен, но клиент не отреагировал как
      ожидалось"
300 echo "Код выхода: $EXIT_CODE"
301 echo "Вывод: $OUTPUT"
302 exit 1
303 fi

```

Д Протоколы тестирования

На рисунке 2 приведен запуск тестов, а также результаты тестирования работы приложения по протоколу TCP, UDP, работы клиента при недоступности сервера.

```
user@computer:~/trpo/lab2/tests$ ./run_all_tests.sh
Запуск всех тестов...
TCP-сервер запущен на порту 8888
тест: работа приложения по протоколу TCP - успешно
Сервер завершил работу.
UDP-сервер запущен на порту 8895
Сервер завершил работу.
тест: работа приложения по протоколу UDP - успешно
тест: работа клиента при недоступности сервера - успешно
```

Рис. 2: Запуск всех тестов

На рисунке 3 приведены результаты тестирования приложения вводом исходных данных с клавиатуры.

```
spawn sh -c g++ -o /home/user/trpo/lab2/tcpserver /home/user/trpo/lab2/tcpserver.cpp -lpthread -std=c++17
spawn sh -c g++ -o /home/user/trpo/lab2/tcpclient /home/user/trpo/lab2/tcpclient.cpp -lpthread -std=c++17
spawn /home/user/trpo/lab2/tcpserver 8910 tcp
spawn /home/user/trpo/lab2/tcpclient 127.0.0.1 8910 tcp
Клиент запущен. Введите 'exit' в любой момент для выхода.

--- Новый запрос ---
Введите количество вершин (V) или 'exit': 6
Введите количество ребер (E): 6
Введите матрицу инцидентности (6 строк, 6 столбцов):
1 0 0 0 0 7
1 2 0 0 0 0
0 2 3 0 0 0
0 0 3 4 0 0
0 0 0 4 5 0
0 0 0 0 5 7
Введите начальную вершину (0-based): 0
Введите конечную вершину (0-based): 5
Кратчайший путь: 0 -> 5
Длина: 7

--- Новый запрос ---
Введите количество вершин (V) или 'exit': тест: ввод исходных данных с клавиатуры - успешно
```

Рис. 3: Результаты тестирования

На рисунке 4 приведены результаты тестирования приложения при вводе исходных данных из файла, тесты на граничные условия и на граф с количеством вершин, равным среднему из ОДЗ. Также приведено сообщение, уведомляющее пользователя о том, что тесты пройдены.

```
тест: ввод исходных данных из файла - успешно
TCP-сервер запущен на порту 8898
Сервер завершил работу.
тест: граф с количеством вершин, на единицу меньше ОДЗ (5) - успешно
TCP-сервер запущен на порту 8903
Сервер завершил работу.
тест: граф с количеством вершин на единицу больше ОДЗ (41) - успешно
TCP-сервер запущен на порту 8920
Сервер завершил работу.
тест: граф с количеством вершин, равным нижней границе ОДЗ(6) - успешно
TCP-сервер запущен на порту 8915
Сервер завершил работу.
тест: граф с количеством вершин, равным верхней границе ОДЗ (40) - успешно
TCP-сервер запущен на порту 8916
Сервер завершил работу.
тест: граф с количеством вершин, равным среднему из ОДЗ (20) - успешно
Все тесты пройдены!
```

Рис. 4: Все тесты пройдены